Trampolining & Stack Safety

Trampoline · Sum 1..100000 without StackOverflowError · Build with Waqar

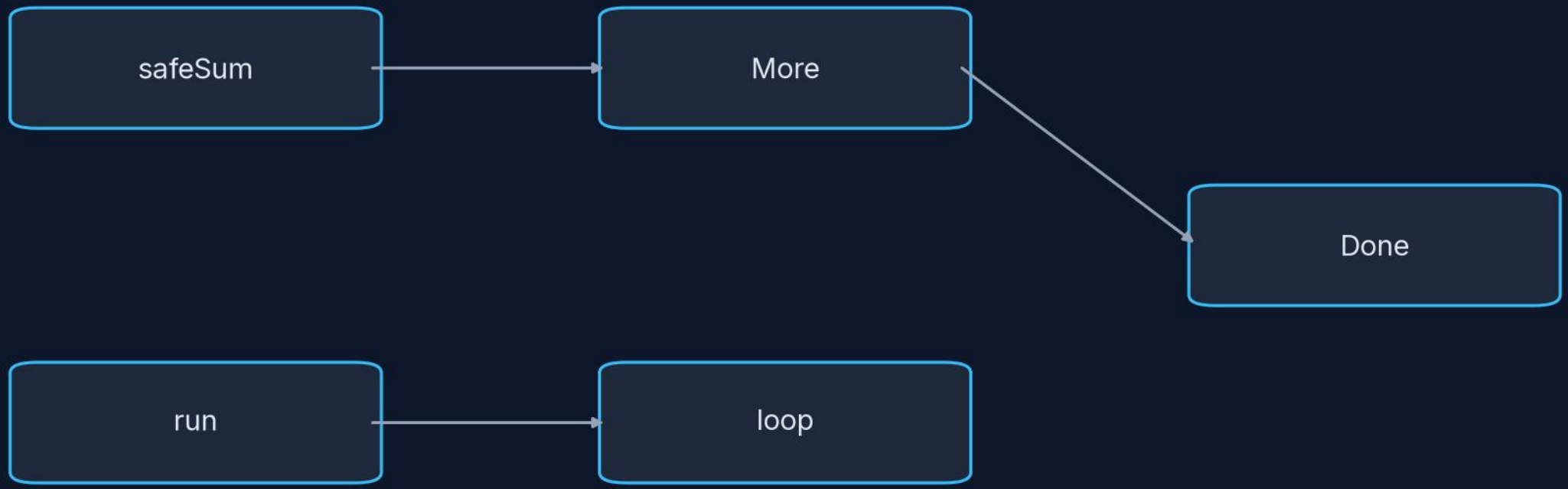
Today's Hook

- Problem: Sum 1..100000 without StackOverflowError
- Pattern: Trampoline — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Sum 1..100000 without StackOverflowError
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Trampoline: Sum 1..100000 without StackOverflowError

Real-World Scenario

- Running example: Sum 1..100000 without StackOverflowError
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.trampoline;

import com.buildwithwaqar.patterns.functional.trampoline.Trampoline;
/** Problem: sum 1..N recursively without StackOverflowError. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Trampoline problem: deep recursion ===");
    System.out.println("sum100000=" + Trampoline.safeSum(100_000, 0).run());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.trampoline;

import java.util.function.Supplier;

/** Stack-safe recursion via trampolining. */
public sealed interface Trampoline<A> permits Trampoline.Done, Trampoline.More {
    record Done<A>(A value) implements Trampoline<A> {}
    record More<A>(Supplier<Trampoline<A>> next) implements Trampoline<A> {}

    static <A> Trampoline<A> done(A a) { return new Done<>(a); }
    static <A> Trampoline<A> more(Supplier<Trampoline<A>> next) { return new More<>(next); }

    default A run() {
        Trampoline<A> cur = this;
        while (true) {
            switch (cur) {
                case Done(var v) -> { return v; }
                case More(var next) -> cur = next.get();
            }
        }
    }
```

Solution (2)

```
    static Trampoline<Long> safeSum(long n, long acc) {
        if (n == 0) return done(acc);
        return more(() -> safeSum(n - 1, acc + n));
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.trampoline;

import com.buildwithwaqar.patterns.functional.trampoline.Trampoline;
/** Problem: sum 1..N recursively without StackOverflowError. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Trampoline problem: deep recursion ===");
    System.out.println("sum100000=" + Trampoline.safeSum(100_000, 0).run());
  }
}
```

Walkthrough Output

```
=== Trampoline problem: deep recursion ===
sum100000=5000050000
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Trampoline.

Related Patterns

- Contrast Trampoline with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Trampoline → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.